ChaCha20-Poly1305 Encryption and Decryption Program

1. Introduction / Background

This project is an encryption and decryption program that uses the ChaCha20-Poly1305 algorithm. The purpose of the program is to demonstrate how plaintext data can be converted into unreadable ciphertext and then restored back into its original form using the correct key and nonce.

ChaCha20 is a symmetric stream cipher, meaning the same secret key is used for both encryption and decryption. It works by generating a complex keystream and combining it with the plaintext to produce ciphertext. Since it is a stream cipher, decryption follows a similar process: the same keystream is generated again and combined with the ciphertext to recover the original plaintext.

For this project, ChaCha20 is paired with Poly1305. Poly1305 provides message authentication, which helps verify that the encrypted data was not changed or tampered with. This makes ChaCha20-Poly1305 more suitable for secure communication because it provides both confidentiality and integrity.

2. Real-World Application of the Algorithm

ChaCha20 is used in modern security systems because it is fast, secure, and efficient. It is especially useful in devices or systems where performance is important. In real-world applications, ChaCha20-Poly1305 is used in secure internet communication, VPN protocols, and other systems that need to protect private data during transmission.

A practical example is secure messaging or secure file transfer. When a user sends a private message, the message can be encrypted using ChaCha20 so that unauthorized users cannot read it. Poly1305 can then verify that the encrypted message was not changed before it reaches the receiver.

3. Planned Algorithm

The program will use the ChaCha20-Poly1305 encryption process.

Encryption Process

1. The user enters a plaintext message.
2. The program checks if the input is valid.
3. The program generates a random secret key.
4. The program generates a random nonce.

5. The plaintext is converted into bytes.
6. The ChaCha20-Poly1305 algorithm encrypts the plaintext.
7. The program outputs the ciphertext, key, and nonce.
8. The key and nonce must be saved because they are needed for decryption.

Decryption Process

1. The user enters the ciphertext.
2. The user enters the key used during encryption.
3. The user enters the nonce used during encryption.
4. The program checks if the inputs are valid.
5. The ChaCha20-Poly1305 algorithm attempts to decrypt the ciphertext.
6. If the key, nonce, and ciphertext are correct, the original plaintext is displayed.
7. If any input is incorrect or the ciphertext was modified, the program displays an error message.

4. Code Breakdown and Explanation

The program will be divided into several parts to make the logic organized and easier to understand.

Importing Required Libraries

The program will import the needed encryption library and other supporting modules. These libraries help generate secure random values, encode data, and perform encryption and decryption.

Generating the Key

The key is a secret value used by the algorithm. The program will generate a random key so that each encryption session is protected. The key must be kept private because anyone with the key can decrypt the message.

Generating the Nonce

The nonce is a unique value used together with the key. It helps ensure that encrypting the same message more than once will not produce the same ciphertext. The nonce does not need to be secret, but it must be saved because it is required for decryption.

Encryption Function

The encryption function accepts plaintext as input. It then converts the plaintext into bytes, encrypts it using ChaCha20-Poly1305, and returns the encrypted output. This part is important because it performs the complex transformation from readable text into ciphertext.

Decryption Function

The decryption function accepts the ciphertext, key, and nonce as input. It uses these values to reverse the encryption process. If the values are correct, the original plaintext is returned. If the values are wrong, the program shows an error instead of crashing.

Error Handling

The program will include error handling to manage unexpected inputs. For example, if the user enters an invalid key, wrong ciphertext, or empty message, the program will display a clear error message. This makes the program more reliable and user-friendly.

5. Sample Input and Output

Sample Encryption

Input:

```
None
Enter message to encrypt: Hello, this is a secret message.
```

Expected Output:

```
None
Encrypted message: 8f4a2c91b7e0...
Key: 1e8f3b7c9a...
Nonce: a92f4c88b1...
```

Sample Decryption

Input:

```
None
Enter encrypted message: 8f4a2c91b7e0...
Enter key: 1e8f3b7c9a...
Enter nonce: a92f4c88b1...
```

Expected Output:

```
None
Decrypted message: Hello, this is a secret message.
```

Sample Error Output

Input:

```
None
Enter encrypted message: wrongdata
Enter key: incorrectkey
Enter nonce: incorrectnonce
```

Expected Output:

```
None
Error: Decryption failed. The key, nonce, or ciphertext may be incorrect.
```

6. Step-by-Step Guide on How to Run the Program

1. Open the executable application.
2. Choose one option from the menu:
    - Encrypt a message
    - Decrypt a message
    - Exit
3. If encrypting, type the plaintext message.
4. Copy and save the generated key, nonce, and ciphertext.
5. If decrypting, paste the ciphertext, key, and nonce into the program.
6. The program will display the decrypted message if the input values are correct.
7. If the input values are wrong, the program will display an error message.

7. Expected Input Format

8. Expected Output Format

For encryption:

```
None
Ciphertext:
Key:
Nonce:
```

For decryption:

```
None
Decrypted message:
```

For invalid input:

```
None
Error message explaining the problem
```

9. Learning / Insights

TBA…

10. References

- RFC 8439: ChaCha20 and Poly1305 for IETF Protocols
- Xilinx / AMD Vitis Security Library: ChaCha20 Algorithm Internals
- Proton: ChaCha20 Encryption Glossary